

CSE 2320 - Homework 8

NAME: Bailey Brown 100/555076

Total points: 100 Topics: merge sort (iterative implementation), hash tables, quicksort, radix sort, count sort, Timsort.

P1 (34 pts) Given files: data1.txt, run1.txt.

Write a program that implements ITERATIVE merge sort. The program should repeatedly read arrays in the format:

N (number of numbers in the array) followed by, on a new line, N numbers separated by spaces. E.g.
8

4 1 9 6 8 1 19 5

When N == -1, the program stops. See data1.txt for the file format.

For each array that it reads, print the original array and then sort it with an ITERATIVE mergesort. To show the behavior of the algorithm, after EACH call to merge, print ONLY THE RECENTLY MERGED section of the array as show in the sample run, run1.txt. **Your program output must match mine exactly**, including the formatted printing of the array. I reserved 5 spaces when printing each array element: `printf("%5d, ", A[i]);`

Save the program in a file called **merge_iter.c**.

P2. (7 points) Is Quick_Sort (as given in the class notes) stable?

If yes, prove it. If no, give an example array, A, (however small or big), sort it with Quick_Sort, and show what the algorithm does that makes it not stable. Use the original array and the final, sorted array to base your proof (do not base your proof on a partially sorted array).

Hint: Focus on the pivot jump.

NO, it is not stable

array A[] = {2a, 2b, 1}; the 2's will swap because 2b > 1

=> array A[] = {2b, 2a, 1};

P3. (7 points) Given the array $A = \langle 8, 6, 9, 2, 7, 1, 5, 10, 6 \rangle$, using the Quicksort Lecture or Figure 7.1, CLRS page 172, as a model, show the execution of the Partition function. Show the $\leq$ partition by **circling the last element of it** and show the $>$ partition by putting a square around the last element of it.

	0	1	2	3	4	5	6	7	8
Original array:	8	6	9	2	7	1	5	10	6
1	8	6	9	2	7	1	5	10	6
2	6	8	9	2	7	1	5	10	6
3	6	8	9	2	7	1	5	10	6
4	6	2	9	8	7	1	5	10	6
5	6	2	9	8	7	1	5	10	6
6	6	2	1	8	9	5	10	6	6
7	6	2	1	5	7	9	8	10	6
8	6	2	1	5	7	9	8	10	6
9	6	2	1	5	6	9	8	10	7

$$\text{pivot} = A[\text{end}] = 6$$

↑
pivot

P4. (9 points) (Radix sort)
Show how **LSD radix sort** sorts the following numbers in the given representation (base 10). Show the numbers after each complete round of count sort.

Index:	0	1	2	3	4	5	6
Original Array:	513	145	320	235	141	433	2
1	320	141	2	513	433	145	235
2	2	513	320	433	235	141	145
3	2	141	145	235	320	433	513

P5. (10 points) **Count sort, radix sort**

Estimate the performance of count sort and radix sort for the following problems and set-ups. Assume you are dealing with numbers on $b=32$ bits. Only positive values are allowed (so there is no sign bit, all 32 bits are used to represent the positive value). Let:

- b – number of bits
- k - number of different possible values of keys
- r – number of bits used for a specific radix
- N – size of array to be sorted.
- d – number of digits in a certain base (on r -bits) representation.

Continue to fill-in the table. Express every term as a power of 2. In the Θ use the values for the terms. E.g. N^2 for $N=8$ would be replaced with $2^6(=64)$. In order to compare quantities easily, we express everything as a power of 2 in the table below (see $7 \approx 2^{2.8}$). See clarification.

N	Count sort			Radix sort for $r = 5$ bits					Optimal radix sort (with optimal r)					Best method	
	k	Θ	Dominant term	k	d	Θ	Dominant term	r	k	d	Θ	Dominant term	Smallest dominant term	Method	
$8=2^3$	2^{32}	$2^3 + 2^{32}$	2^{32}	2^5	$7 \approx 2^{2.8}$	$2^{2.8}(2^3 + 2^5)$	$2^{7.8}$	3	2^3	$2^1 = 2^{0.4}$	$2^{4.4}(2^3 + 2^5)$	$2^{7.4}$	$2^{7.4}$	Radix	
$10^{15} \approx 2^{50}$															

P6. (10 points) **self study of Timsort** Read this article, and answer the Timsort questions below based on it.

(2pt) Timsort was created by: Tim Peters..... It is used as the default sorting algorithm for: Python, Java, Android.....

(3pts) Time complexity: Best case: $\Omega(N)$ Average case: $\Theta(N \log(N))$ Worst case: $O(N \log(N))$

(1pts) What two sorting algorithms does it combine? Insertion Sort + Merge Sort.....

(2 pt) Circle your answer: Is it stable? YES / No Does it do well on arrays with preexisting structure? YES / No

(1 pt) What data does “~sort” indicate in Tim Peters’s introduction to Timsort found here? ... data w/ large masses of equal elements

(1pt) This Wikipedia article discusses a bug found in Timsort. What Java error does that bug produce? ... ArrayOutOfBoundsException

Count array and 0s at [3 after [00e | remove [after another element (see in the copy array)

Index:	0	1	2	3	4	5	6
Copy array:				ETOM		D, SAM	

Index	
Count [array...	D

4

Index...	0
Copy array...	

Index...	D
Copy array...	

P8. (5 points) Hashing

The images below show the occupancy of two hash tables. Both tables have **the same size**, the **same items** hashed in them, and both use **open addressing**. However **they differ** in the way they find an available slot in the table. Which one is a better hash table and why? (You do NOT have to deduce how they find the next available slot. You simply have to judge which one would behave better based on this image.)



The second table is better. There is less clustering, and, therefore, will require less probes to find items, making it more efficient.

P9. (12 points) Hashing

a) (5 points) Give an example of a bad hash function for strings (that generates many collisions). Justify why it is bad: find some strings that will hash to the same cell.

int hash(string s) {
 return strlen(s);
}

Ex: "cat" "dog" "eat" "bug" "bat" "cut" etc.
 will all have a hash value of 3

MANY strings will have the same length

b) (7 points) In the hash table below * and + indicate the cells probed when trying to insert two different items. These items are originally hashed to the same slot (see +* at index 5). The star, *, shows the probed cells for item and the plus, +, shows the probed cells for the other item. You can assume that the table size is very big (e.g. more than 1000) and the slots shown did not require a mod (%) operation (that is we did not have to wrap around).

1) What type of open addressing was used? Justify.

double hashing - the number for index is incremented by a number > 1 every collision

2) Give the next slots to be checked for each item (show where the next * and where the next + will be).

Index	
...	...
5	+ *
...	...
8	+
9	*
...	...
11	+
12	
13	*
14	+
...	...
17	*
...	...

Index	
17	+
...	...
21	*

+ $\Rightarrow$ index + = 3

* $\Rightarrow$ index + = 4

Remember to include your name at the top.

Write your answers in this document or a new document called 2320_H8.pdf. It can be hand-written and scanned, but it must be uploaded electronically. Place 2320_H8.pdf and merge_iter.c in a folder called 2320_HW8, zip that and send it.